

SynthLA: Synthetic Language–Action Multi–task Policies for Zero–shot Real–world Manipulation via Structured Perception

Author Names Omitted for Anonymous Review.

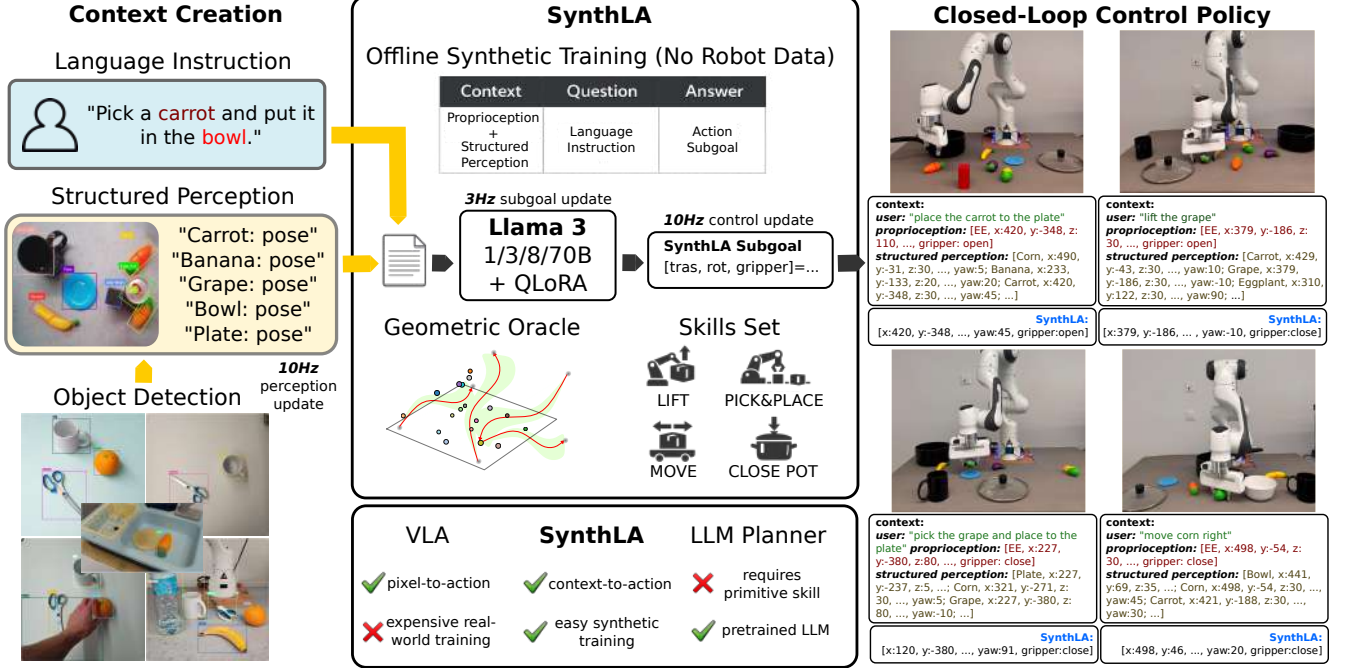


Fig. 1: Overview of SynthLA: RGB-D camera frames of the environment are converted into structured textual descriptions by a YOLO-v8 vision model. We combine scene and robot states with user instructions to obtain a purely textual context. The context is fed to a Llama 3.1 language-action policy that generates low-level robot actions to solve the user task.

Abstract—We introduce SynthLA, a modular framework for learning a low-level language-action policy for robot arms without real-world robot demonstrations or control primitives. We adapt a large language model (LLM) into a language-action policy, whose input is the textual scene context representing the user prompt as well as the environment and robot states. During training, we generate millions of synthetic task-state-action triplets for multiple tasks, leveraging rule-based natural language generation (NLG) of randomized environments. At deployment time, a vision model estimates from RGB-D observations the environment state, used to form the context.

Results indicate that our approach outperforms other SOTA approaches, with a mean success rate of 73% while OpenVLA scores 60% on the same benchmark, despite our approach learning solely from synthetic data. Additional experiments demonstrate that our approach: is robust to out-of-distribution objects and dynamic scenes, is capable of incrementally expanding its task repertoire, and generalizes to unseen objects and novel natural language expressions for the user instructions.

I. INTRODUCTION

Large Language Models (LLMs) trained on vast textual corpora are semantically grounded, i.e., they know common facts about our world, and possess the ability to reason given the current circumstances. Modern language-conditioned approaches leverage the benefits of LLMs to provide prompt-based agents capable of solving multiple, complex tasks. To

this end, recent approaches at one end of the spectrum focus on high-level LLM-based planners capable of composing predefined skills, called primitives, to carry out the task at hand. On the other end of the spectrum, Vision-Language-Action (VLA) models focus on learning low-level skills, attending to a context that combines a text prompt with an image of the environment, going from raw pixels to actions.

Despite their success, LLM-based planners heavily rely on pre-existing primitives that often lack the fine-grained control necessary for complex interactions, and that themselves require training [1]–[6]. Conversely, VLAs learn low-level skills using robot datasets such as Open X-Embodiment [7], assuming access to massive amounts of human demonstrations needed to learn the mapping from raw pixels to actions [8]–[12]. Additionally, VLAs have been found to have poor performance under minor changes in the environment appearance, prompting further human demonstrations to align with the new environment condition [13], [14]. To address these limitations, we propose SynthLA, a modular framework for training multi-task, language-action policies solely from synthetic textual data. The framework is tailored to table-top manipulation with robotic arms, where it is trivial to define actions as cartesian-space trajectories of the end-effector. In detail, we decouple environment sensing from

TABLE I: Comparison of SOTA language-conditioned robot control frameworks used in short-horizon tasks.

Method	Pixel→Action	Modular	Primitive Free	Demonstrations Free	Synthetic Data	Control Level
OpenVLA	✓		✓			Low-level
LLMPlanner (CaP/ProgPrompt)		✓		✓		High-level
LLMPlanner (FLARE)		✓		✓	✓	High-level
SynthLA (ours)		✓	✓	✓	✓	Low-level

task execution, leveraging text as the intermediate representation, or interface, between vision and action models, as shown in Figure 1. A vision model takes the camera feed and predicts the objects present in the scene and their pose relative to the robot. The state of the scene (detected objects and their poses), and the robot (end-effector pose and gripper state) are converted into a structured text and combined with the user command to create the textual scene context. The context is then fed to a language-action policy that predicts the next low-level action in closed loop.

In SynthLA, the language-action policy learns solely from synthetic textual data generated using rule-based Natural Language Generation (NLG) [15]. Specifically, we initialize each episode by randomizing the object present in the scene, their poses, and the pose of the end-effector. We decompose a task into its constituent sub-tasks (e.g., “*close the pot*” becomes go above the lid, move down and grasp, move up, go above the pot, move down and release), and define for each sub-task a function called geometric oracle that produces textual actions. The sub-task of closing the grippers is encoded with `[gripper:close]`, while end-effector movements are done in cartesian space following a linear motion¹ between the current end-effector pose and the sub-task goal pose, e.g., represented as text with the action `[x: 420, y: -348, z: 30, yaw: 40]`. Notably, the vision model is only needed during inference to obtain pose estimates of the real objects seen by the camera, while training involves only the language-action policy.

The framework assumes to know the tasks to be solved, requiring the definition of the set of rules used by the oracle to generate actions, and that the vision model is capable of detecting the required objects. In practice, we expect both assumptions to be reasonable in many scenarios, where the robot tasks rarely change and the objects are well represented in the vision foundation models training data [16], [17]. The three design advantages of SynthLA are: (i) cheaply-generated synthetic training data without the need for human demonstrations or tedious hand-labeling; (ii) reduced input dimensionality and sequence length compared to multimodal VLA training by decoupling perception and action; and (iii) plug-and-play perception supporting arbitrary vision models.

We evaluate SynthLA on four real-world tabletop manipulation tasks (*lift*, *pick&place*, *move*, and *close-the-pot*) with in-distribution and out-of-distribution (i.e., never seen during training) objects and under static or dynamic environments, where an operator moves the objects around and

introduces distractors. Our framework is compared against SOTA baselines on table-top manipulation tasks of known object classes: OpenVLA [9] and Octo [10] for the VLA class, and against CaP [18], ProgPrompt [3] and FLARE [19] as the LLM-based planners. Our goal is not to propose a replacement for VLAs or LLM-based planners, both capable of manipulating novel objects and compose tasks zero-shot, but rather to provide a complementary, tailored solution that is inexpensive to train (primitive and demonstrations free), easily extendable with new objects and tasks, and achieving high success rates across diverse manipulation tasks.

Our **main contribution** is the methodological framework for training language-action policies from synthetic data that are competitive in zero-shot real-world manipulation tasks. The experimental results show that SynthLA: (i) performs comparably well when compared against SOTA baselines, achieving 73% in mean success rate across tasks while a fine-tuned OpenVLA scores only 60%; (ii) is robust to dynamic scenes and out-of-distribution object manipulation, with a moderate 10% decrease in success rate; (iii) learns new tasks incrementally, trading success rates as high as 87% on *pick&place* for being a multi-task policy with 73% success rate on the same task; and (iv) generalizes to different natural language variation of the same tasks, with a moderate drop to 63% in mean success rate. Videos of SynthLA task executions can be found on <https://synthla.github.io>.

II. RELATED WORKS

We can classify language-conditioned control along two distinct dimensions: are perception and control coupled (e.g., raw pixels→actions) or separated? is the control output low-level actions or high-level plans over a set of primitives? Table I categorizes representative approaches and highlights that SynthLA represents a middle-ground: it produces low-level actions like VLAs while avoiding pixel-to-action coupling with a modular interface. Unlike LLM planners, it does not require a hand-engineered skill library and instead learns the language-to-action mapping from synthetic supervision.

A. LLMs for Planning and Skill Composition

LLMs are increasingly used as high-level planners that translate natural language instructions into sequences of executable actions [4]–[6]. A common paradigm is to use the LLM to reason over a predefined library of robot primitives, generating a plan by sequentially composing the primitives, and executing it with the robot. SayCan [1] and follow-ups [2], [20] combine an LLM with a library of primitive robot skills by scoring candidate actions based on their relevance to the instruction and their feasibility in the current

¹Linear motion is the simplest option in our framework; using other interpolators such as higher-order polynomials or Bézier curves is equally possible and will result in smoother end-effector trajectories.

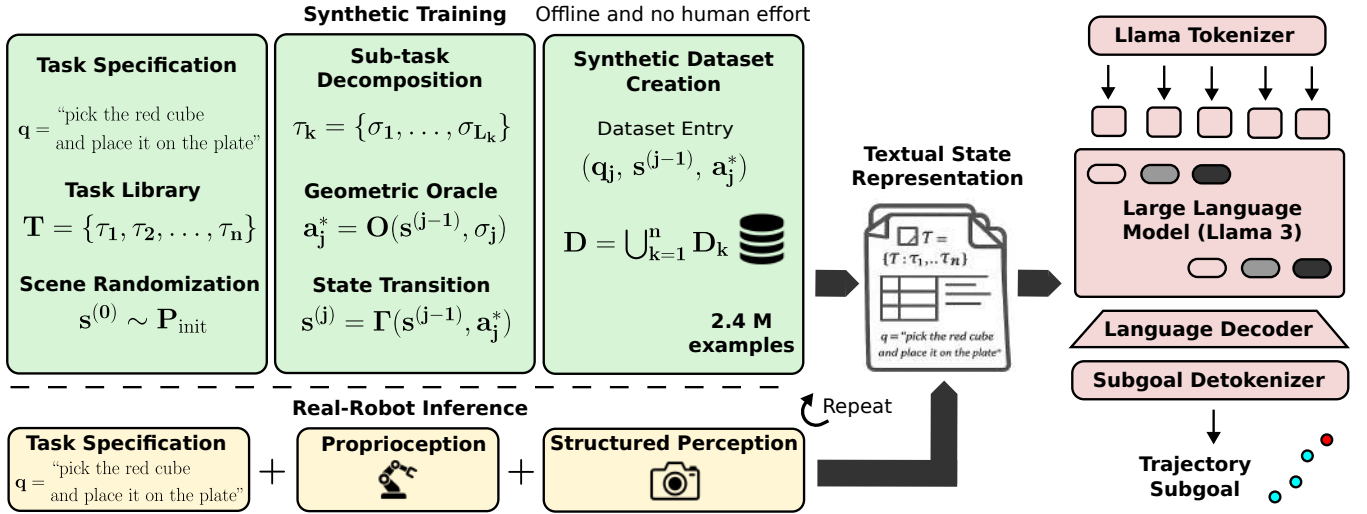


Fig. 2: Given a set of predefined tasks $\mathcal{T}$, we generate synthetic data by randomly sampling an initial environment state $s^{(0)}$ from the distribution of feasible object and robot states P_{init} . Tasks are decomposed into smaller sub-tasks (e.g., move end effector +30cm on the x-axis, open the grippers); each sub-task is then converted into textual actions with rule-based natural language generation (NLG). The state evolves based on the action a_j^* at timestep j according to the state transition function $s^{(j)} = \Gamma(s^{(j-1)}, a_j^*)$; each step yields a task-state-action triplet $(q_j, s^{(j-1)}, a_j^*)$, whose collection forms the training set $\mathcal{D}$ (see Sec. III-A). This strategy enables large-scale, synthetic generation of multi-task demonstrations used to adapt a large language model (LLM) into a language-action policy. During deployment, the environment state is estimated from a vision model and converted into structured text, seamlessly integrating with the language-action policy (see Sec. III-B).

environment. The LLM provides guidance over the task description, while grounding modules ensure that selected actions are actually executable. Instead, other approaches formulate the planning problem as code generation: Prog-Prompt [3] and Code as Policies (CaP) [18] generate executable code that invokes perception and control functions, effectively composing primitives through program synthesis.

SynthLA targets a complementary design point: instead of using language to select among predefined primitives or generate open-loop plans, we learn a *closed-loop* language-action policy that directly outputs low-level robot actions conditioned on a structured scene state. This approach preserves the compositional reasoning capabilities of language models while avoiding reliance on a fixed skill library and enabling continuous feedback during execution.

B. Vision-Language-Action Models

VLAs directly map multimodal observations and language instructions to low-level robot actions by fine-tuning a vision-language model on robotic datasets [8]–[10], [12]. These models demonstrate strong multi-task learning and semantic generalization capabilities across diverse robotic setups, by learning from massive amounts of task demonstrations; for instance, OpenVLA is pretrained on Open X-Embodiment [7], containing nearly one million demonstrations collected across multiple robot platforms. This results in strong visual grounding and language following abilities but makes training computationally demanding and data intensive [9]. Moreover, such models are shown to have limited robustness when deployed outside the training distribution [21], and typically require additional fine-tuning on task-

specific demonstrations [13], [14]. Recent efforts focus on strategies to improve the data efficiency and computational requirements of VLAs. This is done by adopting alternative action representations and decoding strategies to significantly improve inference speed [22], or smaller multimodal backbones with diffusion-based policy heads [23]. In particular, TinyVLA [23] produces competitive manipulation policies by leveraging parameter-efficient adaptation techniques [23].

SynthLA produces low-level robot actions conditioned on language instructions, similarly to VLAs. However, it does so while decoupling perception from action through a structured textual interface (see Figure 2). This is specifically designed for scenarios with a known set of manipulation tasks, enabling policy learning entirely from synthetic supervision while restricting real-world adaptation to the vision model.

C. Real-world Language-conditioned Robotic Datasets

End-to-end pixel-to-action learning commonly relies on large real-robot demonstration datasets [7], [24], which are costly to collect and difficult to scale across tasks and environments [7], [23]. Several robotics datasets have been proposed for grasping, demonstrations, and interaction learning [25], [26]. While some datasets span multiple robot platforms, such as RoboNet [27] and Open X-Embodiment [7], most require substantial human involvement and real-world data capture, which makes them resource-intensive to expand and to maintain. Instead, SynthLA uses a policy whose input is structured text, supporting synthetic data generation without images, task demonstrations, or tedious hand-labeling.

III. METHODOLOGY

Our goal is to learn a language-conditioned low-level policy across a set of n manipulation tasks using *only synthetic supervision*. Let the set of tasks be $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$; each task τ_k is defined by an instruction space $\mathcal{Q}_k$, a state space $\mathcal{S}_k$, and an action space $\mathcal{A}_k$. We represent a state $s \in \mathcal{S}_k$ as $s = (s_{\text{robot}}, s_{\text{env}})$, where $s_{\text{robot}} = (\mathbf{p}_{ee}, g)$ contains the end-effector pose $\mathbf{p}_{ee} \in \mathbb{R}^4$ (position and yaw) and the gripper state $g \in \mathcal{G}$ (open/closed). The environment state is $s_{\text{env}} = \{(c_i, \mathbf{p}_i, \theta_i)\}_{i=1}^M$, where c_i is object class, $\mathbf{p}_i \in \mathbb{R}^3$ is object position, and θ_i is yaw orientation.

We learn a multi-task language-conditioned policy $\pi : \mathcal{Q} \times \mathcal{S} \rightarrow \mathcal{A}$ predicting the next low-level action given instruction-state pairs (q, s) , where $\mathcal{Q} = \bigcup_{k=1}^n \mathcal{Q}_k$ and $\mathcal{S} = \bigcup_{k=1}^n \mathcal{S}_k$. After generating synthetic, task-specific data (Sec. III-A), we adapt a language model into a language-action policy with QLoRA [28] (Sec. III-B).

A. Synthetic Dataset Generation

Training data consisting of actions and structured synthetic states are produced by generators. For each task $\tau_k \in \mathcal{T}$ we define a generator $G_k : \mathbb{Z}^+ \rightarrow \mathcal{Q}_k \times \mathcal{S}_k \times \mathcal{A}_k$ that produces triplets (q, s, a^*) , where q is a valid language instruction, s is a synthetic state, and a^* is the target low-level action. Specifically, each task τ_k is decomposed into an ordered sequence of L_k sub-tasks, $\tau_k = \{\sigma_1, \sigma_2, \dots, \sigma_{L_k}\}$.

Episodes begin in a random initial symbolic state $s^{(0)}$ distributed as $\mathcal{P}_{\text{init}}$, the distribution of feasible robot and object configurations. Synthetic textual actions are generated based on the state using geometric oracle functions that implement linear movements of the end effector according to the current sub-task. Oracles use handcrafted rules to generate structured text with natural language generation (NLG) [15]. Take as an example the task “*pick a carrot and put it on the plate*”: it is decomposed into move the end effector above the carrot, go downwards, close the gripper, move upwards, go above the plate, move downwards, and open the gripper. Actions consist of a text encoding the movement towards the next sub-task pose, or open/close the gripper; benchmark tasks are shown in Figure 3. We assume that the future state after executing the current action is deterministically defined by the action itself $s^{(j)} = \Gamma(s^{(j-1)}, a_j^*)$, so as to mimic a perfect execution. Each generated action is appended to the current episode until task completion. The complete dataset $\mathcal{D}$ is the collection of episodes generated across all tasks.

Importantly, the oracle operates *locally* at the level of sub-tasks: it does not solve global planning, but rather provides a consistent expert action that advances the execution toward sub-task completion. This design makes data generation scalable; large datasets can be generated autonomously by sampling initial scenes $s^{(0)} \sim \mathcal{P}_{\text{init}}$ and querying the oracle to get task-state-action triplets.

B. Training and Deployment

During training, we adapt a pretrained LLM into a low-level policy $f_\theta : \mathcal{Q} \times \mathcal{S} \rightarrow \mathcal{A}$ on the synthetic dataset $\mathcal{D}$ using QLoRA [28], which freezes quantized base weights and



Fig. 3: In our experiments, we consider four table-top manipulation tasks: (top-left) close the pot with the lid, (top-right) drag the object on the table’s surface, (bottom-left) lift the object up, and (bottom-right) pick&place.

learns low-rank adapters. We optimize next-action prediction with teacher forcing over the action token sequence. This supervision is generated at scale without real-robot demonstrations, unlike large-scale imitation learning pipelines that rely on collecting teleoperated image-action trajectories.

At deployment, a vision model ϕ maps raw RGB-D observations $o \in \mathcal{O}$ into a structured environment state, as shown in the bottom of Figure 2. The environment state is extracted from one or multiple, possibly heterogeneous cameras. For each camera $c \in \mathcal{C}$, the perception module ϕ_c processes observations $o_t^{(c)}$ and outputs object detections in image space:

$$\phi_c(o_t^{(c)}) = \text{obj}_{\text{px}}^{(c)} = \{(c_i, \mathbf{p}_{\mathbf{x}_i}, d_i, \alpha_i)\}_{i=1}^M,$$

where c_i is the object class, $\mathbf{p}_{\mathbf{x}_i} \in \mathbb{R}^2$ is pixel location, d_i is depth, and α_i is orientation. Using camera calibration, detections are converted into world-frame poses: $s_{\text{env}}^{(c)} = \{(c_i, \mathbf{p}_i, \theta_i)\}_{i=1}^M$. We fuse $s_{\text{env}}^{(c)}$ across cameras into a unified s_{env} as the union of detections of all cameras, breaking ties by assigning a priority to each camera.

Given the current state, our policy predicts *absolute* target references (target pose + gripper command) at approximately 3 Hz, while a higher-frequency low-level controller continuously tracks the latest target. Thanks to the target reference updates being asynchronous with respect to servo control, motion remains continuous and is corrected online as the scene evolves over time, adapting to changes. By contrast, end-to-end VLA models typically output per-step *delta* actions from the latest image-language inference, so correction is coupled to the next full inference cycle.

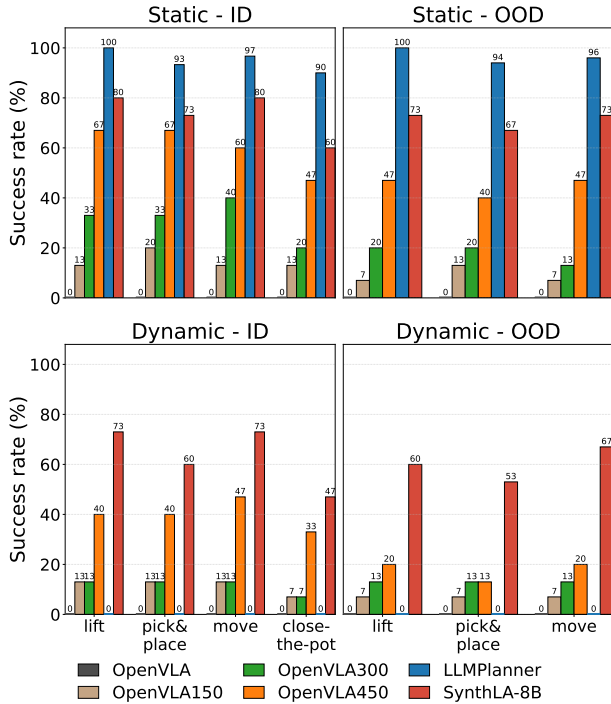


Fig. 4: Comparison between SynthLA and OpenVLA variants across static and dynamic environments. Each subplot reports success rate (%) over the four manipulation tasks. SynthLA achieves higher performance in structured settings without requiring real-world training trajectories.

IV. EXPERIMENTAL RESULTS

We evaluate SynthLA on four real-world tabletop manipulation tasks under two execution regimes: in the *static* setting, objects in the scene are fixed, while in the *dynamic* setting, scene objects are displaced and additional distractors enter the scene during execution. We also consider *in-distribution* (ID) objects that are present in the synthetic training set and *out-of-distribution* (OOD) objects, i.e., novel objects not seen during training. Our evaluation addresses five questions:

- 1) Can the synthetic training of SynthLA enable real-world, zero-shot manipulation?
- 2) How robust is SynthLA to out-of-distribution objects and dynamic environments?
- 3) How do different backbone sizes affect latency and performance?
- 4) Is SynthLA robust to variations in language, perception, and physical interaction?
- 5) How does the addition of tasks impact the overall performance?

A. Experimental Setup

As the vision model, we use a YOLO-v8 OBB model fine-tuned on a custom dataset with images and associated object poses, running at 10 Hz on an RTX 4090. The model takes as input the feed of a single Intel RealSense camera [29] mounted externally, capturing the whole scene and achieving 95% detection accuracy on known categories. The model we

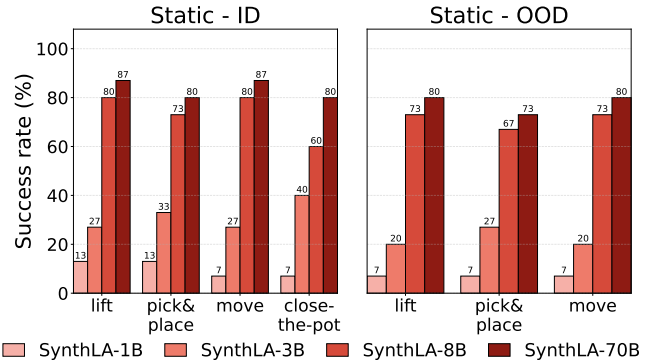


Fig. 5: Performance of different sizes of the Llama 3.1 backbone in static scenes. Larger models consistently improve performance at the cost of an increased inference time.

TABLE II: SynthLA runtime and computational cost.

Model	Inf. time [s]↓	Training time [h]↓	#GPU↓
1B	0.10	2	1
3B	0.19	7	1
8B	0.33	16	1
70B	1.62	48	8

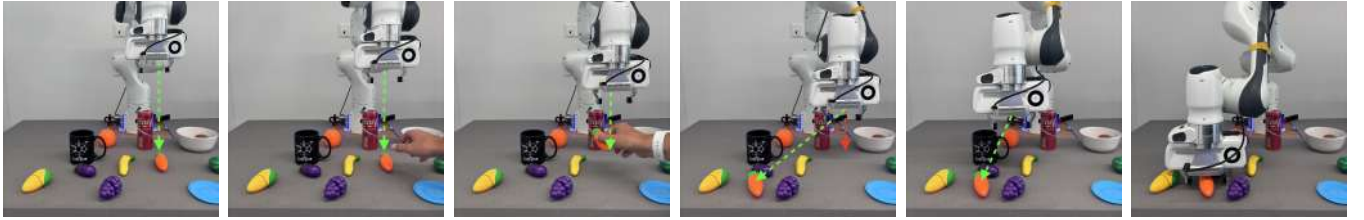
used for the experiments reported below yields an average position error of 3.1 mm (variance 0.49 mm^2) on 20 trials, highlighting the benefits of a task-specific vision module decoupled from the policy. We also evaluated the approach with a pretrained YOLO-v9 model on our dataset without fine-tuning on the specific objects, where the model showed comparable performance on common household objects such as fruits, office items, cups, scissors, bottles and robustness to distractors and operator hands moving objects around. For the language-action policy f_θ , we adopt Llama 3.1 with 8B parameters [30], running on a single RTX 4090 at 3 Hz.

Training of SynthLA is done exclusively on synthetic data, by adapting Llama into a language-action policy with a dataset consisting of 2.4M triplets (600k per task). Training is performed on a single RTX 4090 for approximately 16 hours. In contrast, OpenVLA is fine-tuned on 450 demonstrations per task, collected by teleoperation with a SpaceMouse over 60 hours and totaling 1800 task demonstrations.

B. Baselines and Evaluation Protocol

We evaluate four manipulation tasks: *lift*, *pick&place*, *move*, and *close-the-pot* (Fig. 3). Performance is measured by task success rate over 15 trials per task under each condition (static or dynamic scenes, and in-distribution or out-of-distribution objects).

We test four variants of SynthLA using Llama 3.1 backbones with 1B, 3B, 8B, and 70B parameters. As a pixel-to-action baseline, we compare against OpenVLA [9], evaluating both the pretrained zero-shot model and versions fine-tuned with QLoRA on teleoperated demonstrations (VLA150, VLA300, VLA450), corresponding to 150, 300, and 450 demonstrations per task. We also compare against FLARE [19], an LLM-based planner using Llama 3.1, re-



(a) SynthLA generates an action to reach the carrot. (b) Executing the action. (c) The carrot's position changes. (d) SynthLA computes a new action. (e) Approaching the carrot at its new position. (f) Carrot reached.

Fig. 6: The model is instructed to “*pick the carrot and place it in the bowl.*” SynthLA initially plans a trajectory (Fig. 6a), but during execution, the carrot is displaced (Figs. 6b–6d). Within 0.33 s, SynthLA recomputes a trajectory from the updated state (Figs. 6e–6f), demonstrating real-time adaptation in a dynamic environment.

ferred to as *LLMPlanner*. For short-horizon manipulation tasks, different planner frameworks (e.g., CaP [18] or Prog-Prompt [3]) show comparable behavior, as failures typically arise from perception errors or primitive execution rather than planning itself. LLMPlanner produces high-level plans composed of predefined primitives executed by the same low-level controller used in SynthLA.

Finally, we evaluate generalization by comparing SynthLA, LLMPlanner, OpenVLA, and Octo [10] across several perturbation axes. We report the success rate of each method under motion, semantic, language, visual, and physical perturbations, measured relative to its baseline performance on the pretraining distribution.

C. SynthLA is Robust to OOD Objects and Dynamic Scenes

In static scenes, top row of Figure 4, SynthLA achieves competitive success rates both in ID (mean success rate across tasks of 73%) and OOD (71%) object manipulation despite having been trained only on synthetic trajectories. OpenVLA, in contrast, performs subpar consistently across all tasks and all amounts of fine-tuning demonstrations, with OpenVLA450 achieving a mean success rate of 60% ID and 45% OOD. We attribute the gap in OpenVLA’s success rate to its limited data efficiency, where substantially more than 1800 demonstrations are needed for a comparable performance. LLMPlanner outperforms SynthLA across all tasks in static environments, achieving an average success rate of 95% ID and 96% OOD. The sporadic failures are primarily attributable to inaccuracies in object pose detection rather than planner failure. In terms of tasks, *close-the-pot* is the most challenging, requiring the robot to grasp the lid handle and precisely align it with the top part of the pot.

In dynamic scenes, we repeat our evaluation while perturbing object positions and introducing distractors during task execution (see Figure 6). In the bottom row of Figure 4, we show that SynthLA is robust to this scenario thanks to a modular design, leveraging modern vision models that ignore distractors and detect partially occluded objects, scoring 63% ID and 60% OOD (-10% success rate compared to static scenes). Conversely, OpenVLA is more sensitive to distractors in its field of view (mean success rate of 18% OOD); we note that even the operator’s hand moving ID

objects is enough of a domain shift to cause OpenVLA to fail frequently (40% ID). The LLM planner fails in dynamic scenes due to the open-loop plan execution, without noticing that the target object has been moved, and sometimes generating imprecise plans when distractors are present.

D. SynthLA 8B Balances Efficiency and Performance

We evaluate Llama backbones of increasing size (1B, 3B, 8B, 70B) in Figure 5, showing the success rates obtained by evaluating these variants with the protocol defined in Sec. IV-B. In Table II, we report the inference latency, training time, and number of GPUs used in the experiment. Results indicate that performance improves with model size, albeit with diminishing returns, while latency and training cost increase. The 8B model provides the best trade-off between performance and inference speed, while the 70B model achieves the highest success rates, though it requires 8 GPUs to fit the parameters into memory.

E. Different Approaches Have Complementary Strengths

TABLE III: Example instructions used for the language and semantic generalization experiment on the *pick&place* task.

Language and Semantic Generalization		
Variation	Example Task Sentences	
Verb variation	“Transfer Obj1 to Obj2”	“Lift Obj1 into Obj2”
Synonyms	“Place Obj1 inside Obj2”	“Move Obj1 to Obj2”
Reformulation	“Put Obj1 in Obj2”	“Into Obj2, place Obj1”
Multilingual	“Put Obj1 in Obj2” (EN)	“Poner Obj1 en Obj2” (ES)

We compare SynthLA and the SOTA baselines across a wide range of generalization tasks, as shown in Figures 7–8, highlighting complementary strengths of the different approaches. LLMPlanner obtains the highest scores on *motion*, *semantic*, and *language*-grounding tests when the primitive library is well matched to the workspace; in short-horizon settings, an off-the-shelf LLM planner is highly effective, while SynthLA always comes in second place. Examples of semantic generalization instructions are reported in Table III. On *visual* generalization, LLMPlanner has the same relative success rate as SynthLA because performance depends on the

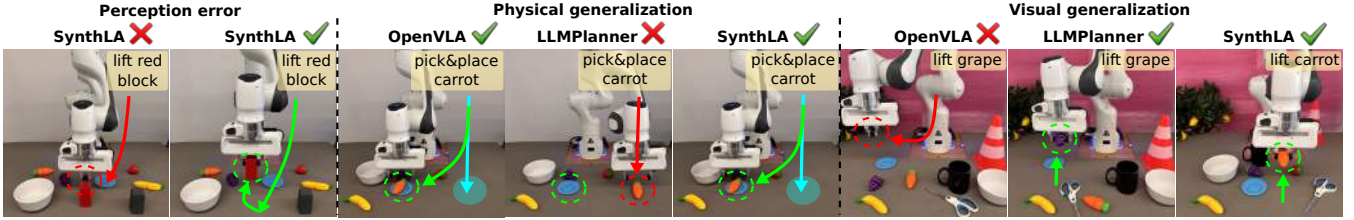


Fig. 7: Qualitative comparison across representative generalization scenarios. From left to right: perception error, physical generalization (i.e., when the light blue plate is moved left by the user during execution), and visual generalization. The examples highlight typical behaviors of the evaluated approaches. SynthLA successfully adapts to changes in object pose and scene conditions thanks to its closed-loop policy, while end-to-end pixel-to-action models are more sensitive to perception errors and LLM planners may fail when physical conditions deviate from the assumed primitive execution.

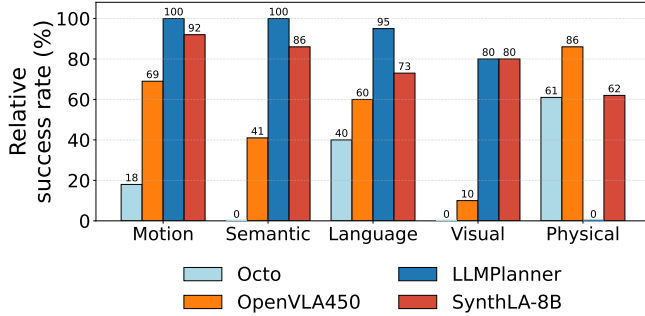


Fig. 8: Robustness under *motion* (unseen object positions), *semantic* (unseen instructions and concepts), *language* grounding (manipulating objects from prompts), *visual* (noisy backgrounds and distractors), and *physical* (unseen trajectories, shapes and scenes) perturbations. Relative success rate measures the per-model relative change in success rate compared to no perturbations (100% = same success rate, 50% = success rate halved, 0% = total failure).

same vision model, while end-to-end VLA variants degrade more under background and distractor changes (e.g., new objects or a human hand in the scene).

Conversely, SynthLA and VLA methods are stronger on low-level action generalization due to being able to run in closed-loop, whereas the plan of an LLM planner cannot react to environmental changes. Representative qualitative examples of these behaviors are shown in Figure 7, highlighting typical perception errors and differences in physical and visual generalization across approaches. This difference reflects the underlying design trade-off in language-vision-action fusion: SynthLA exhibits stronger generalization on the language side, while VLA models tend to generalize better on the action side due to their direct vision-to-action mapping. From a physical generalization perspective, the language component in SynthLA remains largely ID, whereas the resulting actions often operate in OOD physical conditions. OpenVLA and Octo models instead learn a direct vision-action mapping that, while sometimes less precise, tends to generalize better across unseen actions.

SynthLA performs consistently well across multiple generalization axes and remains robust when the skill set is well

defined. Unlike end-to-end VLAs, our approach focuses on a smaller set of manipulation tasks and does not aim to learn a generalist vision-action mapping. Instead, the modular design decouples perception from control, enabling robust visual grounding while leveraging the reasoning capabilities of large language models for closed-loop action generation. This separation allows SynthLA to execute manipulation tasks zero-shot using only synthetically generated textual supervision, without requiring real-robot demonstrations.

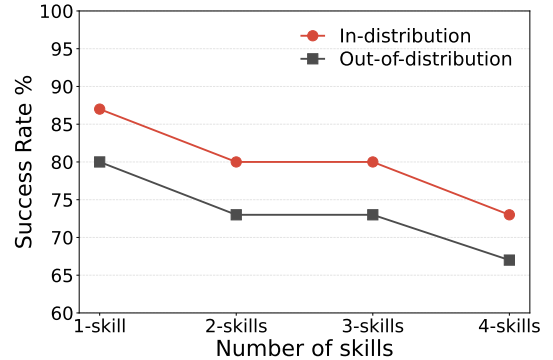


Fig. 9: Impact of new skills acquisition on previous learned skills with SynthLA-8B. We investigate the performance drop in the pick&place task, that is the first skill acquired.

F. SynthLA Can Incrementally Expand Its Task Repertoire

We analyze the effect of increasing task diversity in static scenes in Figure 9, starting with the 8B variant of our language-action model trained only on *pick&place* (1 skill). Extending SynthLA to a new task consists of specifying (i) the state variables needed, (ii) the prior distribution of robot and object states, and (iii) a geometric oracle implementing the required sub-tasks². This produces new synthetic task-state-action triplets that are appended to $\mathcal{D}$ and used to update the same language-action policy via parameter-efficient fine-tuning, without collecting real robot trajectories.

We adapt the model to a larger task repertoire, incrementally adding *lift*, then *move*, and finally *close-the-pot*, keeping

²Sub-tasks such as open/close the grippers, and move the end-effector in cartesian space need to be defined only once, and can be re-used for multiple manipulation tasks. Only new sub-tasks require engineering effort.

the number of synthetic samples per task constant. As the number of tasks increases, *pick&place* performance drops moderately both in ID, going from 87% to 73%, and in OOD, going from 80% to 62%. The results highlight the trade-off between a specialist and generalist policy, with the latter sacrificing performance to be able to solve diverse tasks.

V. LIMITATIONS

SynthLA is tailored for table-top manipulation tasks with robot arms, and targets scenarios where tasks and objects of interest are known in advance. As such, one limitation of the framework is that generating synthetic supervision requires task specification, by decomposing tasks into parameterized sub-tasks and implementing the required geometric oracles. While oracles can be reused for different tasks, there is a one-time engineering effort to define the set of rules for new oracles, requiring domain expertise that is in stark contrast with pure demonstration-based learning. Additionally, SynthLA assumes a closed set of detectable object categories, limiting its application to settings with common objects, or where fine-tuning data can be collected for adapting the vision model. The runtime inference of SynthLA is bottlenecked by the Llama backbone, which suffices for the considered tabletop tasks but may be inadequate for faster dynamics. Lastly, our evaluation focuses on short-horizon manipulation tasks; extending the approach to longer-horizon tasks, heavier clutter, and richer contact dynamics likely requires richer state representations and more expressive oracles.

VI. CONCLUSION

We presented SynthLA, a modular framework for low-level, language-conditioned manipulation that leverages a structured text representing the scene, generated from a vision model, and a learned language-action policy. Training is done entirely using synthetic data generated by a geometric oracle with rule-based natural language generation. Using text as the interface between vision and action enables scalable skill acquisition without real-robot trajectories, efficient fine-tuning, and real-time closed-loop deployment on consumer hardware. Across four real-world tabletop manipulation tasks under static and dynamic scenes with in-domain and out-of-domain objects (see Section IV), SynthLA has a strong performance while requiring no costly demonstrations to train the policy. Overall, the results indicate a practical trade-off: compared to end-to-end VLAs, structured modular perception improves data efficiency and simplifies adaptation at the cost of reduced open-world vision-action generality; however, the system still performs strong physical grounding and differs from language-based planners by operating directly at the low-level control layer in closed loop, without relying on predefined primitives.

In future work we will explore: smaller backbones and decoding strategies to increase control frequency, use of open-vocabulary vision models for uncommon objects, and expanding the skill repertoire while improving task retention.

REFERENCES

- [1] M. Ahn et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [2] R. Hazra et al. Saycanpay: Heuristic planning with large language models using learnable domain knowledge. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [3] I. Singh et al. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11523–11530. IEEE, 2023.
- [4] K. Lin et al. Text2motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47(8):1345–1365, 2023.
- [5] W. Huang et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- [6] D. Shah et al. Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action. In *Conference on robot learning*, pp. 492–504. PMLR, 2023.
- [7] A. Padalkar et al. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2023.
- [8] A. Brohan et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [9] M. J. Kim et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [10] Octo Model Team et al. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024.
- [11] A. Brohan et al. Rt-1: Robotics transformer for real-world control at scale. *Proceedings of Robotics: Science and Systems*, 2022.
- [12] K. Black et al. pi0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [13] J. Zhou et al. Exploring the limits of vision-language-action manipulations in cross-task generalization. *arXiv preprint arXiv:2505.15660*, 2025.
- [14] X. Li et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.
- [15] C. Dong et al. A survey of natural language generation. *ACM Comput. Surv.*, 55(8), 2022.
- [16] A. Sun et al. Explain any concept: Segment anything meets concept-based explanation. *Advances in Neural Information Processing Systems*, 36:21826–21840, 2023.
- [17] M. Oquab et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.
- [18] J. Liang et al. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation*, pp. 9493–9500, 2023.
- [19] M. Maccarini et al. Flare: Fine-tuned large language models for resource-efficient action generation in robotics. *Procedia CIRP*, 134:759–764, 2025.
- [20] W. Huang et al. Guiding text generation with grounded models for embodied agents. In A. Oh et al., editors, *Advances in Neural Information Processing Systems*, volume 36, pp. 59636–59661, 2023.
- [21] S. Zhang et al. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. *arXiv preprint arXiv:2412.18194*, 2024.
- [22] M. J. Kim et al. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [23] J. Wen et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2409.12514*, 2024.
- [24] A. Khazatsky et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [25] C. Finn and S. Levine. Deep visual foresight for planning robot motion. In *IEEE International Conference on Robotics and Automation*, pp. 2786–2793, 2017.
- [26] D. Kalashnikov et al. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *CoRR*, abs/2104.08212, 2021.
- [27] S. Dasari et al. Robonet: Large-scale multi-robot learning. *arXiv preprint arXiv:1910.11215*, 2019.
- [28] T. Dettmers et al. Qlora: Efficient finetuning of quantized llms. In A. Oh et al., editors, *Advances in Neural Information Processing Systems*, volume 36, pp. 10088–10115, 2023.
- [29] L. Keselman et al. Intel realsense stereoscopic depth cameras. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pp. 1–10, 2017.
- [30] A. Dubey et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.